

Numerical Linear Algebra Project

Choose one of the three problems below. Do not do all three. You will only be graded on one problem. The one you want me to grade.
Please ask me by email if directions below are not clear.
Codes need to be in “julia”.

Logistic

Three topics to choose from:

1. NEP – nonsymmetric eigenvalue problem: Given an n -by- n real nonsymmetric matrix, A , compute “quasi Schur form”: $A = QTQ^T$.
2. SEP – symmetric eigenvalue problem: Given an n -by- n real symmetric matrix, A , compute $A = QDQ^T$.
3. SVD – singular value decomposition: Given an m -by- n real matrix, A , compute $A = USV^T$.

Choose whichever one you prefer; more information on the topics is below. For SVD, the book by Darve and Wooters is very well done (Section 5.4). In an ideal world, you follow the book and you’re good to go. For NEP, there’s already a lot done in `./src/gees.jl` in Eric Darve’s GitHub; obviously, that being said it’s not just a matter of copying and pasting the code and submitting it to me. Yes, you can reuse everything. And I encourage you to look at everything carefully, but there’s extra work to do. For SEP, you need to “symmetrize NEP.” See Darve and Wooters’ book too. For SEP and SVD, the problems are intrinsically simpler to solve, but you’re starting from scratch. For NEP, you have some building blocks in Darve’s GitHub.

Please deliver as well some tests and some examples of usage of your code. It will be important that you assert that your code is correct and usable. That way you will also have to demonstrate your understanding of the properties of the input and output of your code.

The projects must be done individually. You can communicate with each other, but no copying code. I didn’t have any problems with cheating on assignments 1, 2, and 3, so I’m not worried about the end-of-semester projects. Keep up the good work.

Deliverable: The written report is simply your code. Either a `.jl` file or a `.ipynb` file. (Or a mix.) No need to add a PDF, etc. I need to be able to execute your code.

The grade will be a “holistic” grade, so it’s not 10% for the oral presentation and 40% for the written exam, but it’s a grade for everything. Ultimately, what matters most are the codes, you need to be able to explain the codes clearly during the oral

presentation.

Project #1: NEP, nonsymmetric eigenvalue problem

In input a nonsymmetric n -by- n matrix A . In output the Schur form of A . ($A = QTQ^T$.) Only real arithmetic, no complex arithmetic. T is quasi triangular so 1x1 block for real eigenvalues and 2x2 block for complex eigenvalues. Please have a look at `gees.jl` in `./src` of GitHub of Eric Darve for a template. Algorithms needs to be the implicitly shifted QR iteration algorithm using Francis shift. Please first reduce to Hessenberg form, then each iteration needs to be $\mathcal{O}(n^2)$ and so needs to take into account the Hessenberg structure of H . Three things not done in `gees.jl` that you need to do (1/3) Need flags for either 'D' compute the diagonal only, 'T' compute the Schur form only, or 'V' compute both Q and T . (2/3) Need to fix the "explicit shift strategy". (3/3) Return the 2x2 block (whenever they exist on the diagonal) as "standard" form. See DM3 question 3.

Goal is that your code is able to work on a random matrix. Then you can try on a matrix of the type $A = VDV^{-1}$ when V is ill conditioned. Also you can try with cluster of eigenvalues. A few other types of matrices are interesting.

Analysis: (*) please provide an example where we observe quadratic convergence for bottom right side of matrices, (*) return a metric such as "number of QR iterations per eigenvalue", (*) time your code for various n and try to scale up to larger n (say up to size 300), compare with native julia functionality.

Rubrique: your code works for random matrices, is clearly written and the analysis (stability, convergence and timing) is done, the choices made in the design of the code are clearly explained in the comments of the code, you are able to clearly explain the code and answer questions during the oral presentation.

Project #2: SEP, symmetric eigenvalue problem

In input a symmetric n -by- n matrix A . ($A = A^T$.) In output the eigendecomposition of A . ($A = QDQ^T$.) D is diagonal real and Q is n -by- n orthogonal: $Q^TQ = QQ^T = I$. Only real arithmetic, no complex arithmetic. (So do not take A as complex Hermitian ($A = A^H$).) Real symmetric is enough. First step is to take A and reduce to symmetric tridiagonal form. You can only use the lower part of A . (Or the upper, as you wish. But only one half.) I recommend starting with `gehrd!` and then adapting for the symmetric case. But as you wish. The nonsymmetric QR algorithm that is explained in Darve and Wootters needs to be "symmetrized". This is not that easy. Your final algorithm should only work on one half of a matrix, use $n^2/2 + \mathcal{O}(n)$ memory, and perform $4/3n^3 + \mathcal{O}(n^2)$ FLOPS.

Need flags for either ('D') compute the diagonal only or 'V' compute both Q and

D. The goal is to perform $\mathcal{O}(n)$ operations per QR step if eigenvalues only are requested, and $\mathcal{O}(n^2)$ operations per QR step if eigenvalues and eigenvectors are requested.

Analysis: (*) please provide an example where we observe cubic convergence for bottom right side of matrices, (*) return a metric such as “number of QR iterations per eigenvalue”, (*) time your code for various n and try to scale up to larger n . (say up to size 300), compare with native julia functionality.

Rubrique: your code works for random matrices, is clearly written and the analysis (stability, convergence and timing) is done, the choice made in the design of the code are clearly explained in the comments of the code, you are able to clearly explain the code and answer questions during the oral presentation.

Project #3: SVD, singular value decomposition

In input an m -by- n matrix A . (Please take $m \geq n$.) In output the thin SVD of A . ($A = USV^T$.) Only real arithmetic, no complex arithmetic. Please read section 5.4 in Darve and Wooters. The algorithm is explained there.

Analysis: (*) please provide an example where we observe cubic convergence for bottom right side of matrices, (*) return a metric such as “number of QR iterations per singular value”, (*) time your code in the square case for various $m = n$ and try to scale up to larger $m = n$ (say up to size 300), compare with native julia functionality.

Rubrique: your code works for random matrices, is clearly written and the analysis (stability, convergence and timing) is done, the choice made in the design of the code are clearly explained in the comments of the code, you are able to clearly explain the code and answer questions during the oral presentation.